

MODULE 5.1 · Concept 3 of 8

Simple Reactive Agents

Condition → Action rules with zero state — the simplest agent architecture

8 min

Duration

B2 – Understand

Bloom's

L2 – Intermediate

Level

TH-000002

Prerequisite

AI-AG-FN-TH-000003

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHAT

Stimulus-Response Agents

No memory, no planning — just IF → THEN rules

Formal Definition: $\text{action} = \text{rules}(\text{percept})$

The simplest possible agent: a direct mapping from percept to action

Definition: A reactive agent selects actions based solely on the **current percept** — no internal state, no history, no planning. It implements a pure function:

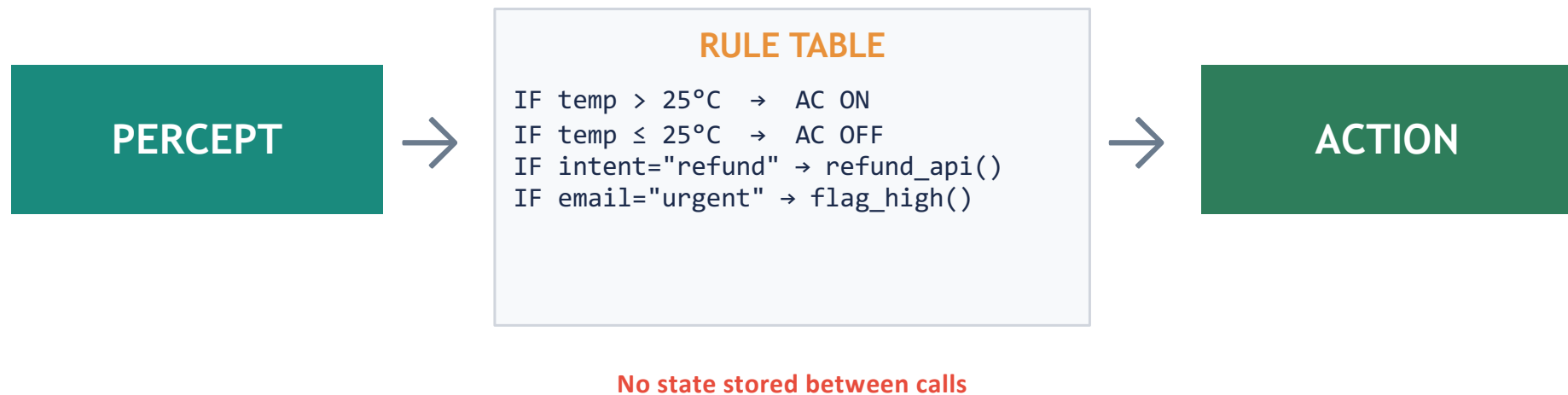
$$\text{action} = \text{rules}(\text{percept})$$

What this means concretely:

- The agent receives a single input (percept) and immediately produces an output (action)
- No reference to ANY previous interaction — each percept is handled independently
- The rules are fixed at design time — they do not change based on experience
- The function is deterministic: same percept always produces same action
- No intermediate "thinking" step — input maps directly to output

Stimulus-Response Mapping

Visual: how a percept flows through the rule table to produce an action



The rule table is static. It does not learn, adapt, or remember. First match wins.

Rule Examples in Detail

Three domains — same pattern: IF condition THEN action

Smart Thermostat

```
IF sensor.temp > 25:  
    turn_on(AC)  
ELIF sensor.temp < 20:  
    turn_on(heater)  
ELSE:  
    standby()
```

Runs every 30 seconds. No memory of how long AC has been on. No weather forecast.

Email Priority Filter

```
IF "urgent" in email.subject:  
    flag_priority("high")  
ELIF sender in VIP_list:  
    flag_priority("medium")  
ELSE:  
    flag_priority("normal")
```

Keyword match only. Can't understand context or detect sarcasm.

Intent Router

```
IF classify(msg) == "refund":  
    route_to(refund_agent)  
ELIF classify(msg) == "track":  
    route_to(tracking_agent)  
ELSE:  
    route_to(general_agent)
```

Classification is reactive — same input always routes to same handler.

Traced Walkthrough: Smart Thermostat

Step by step — notice what's missing

Time	Percept	Rule Matched	Action	Note
10:00 AM	Sensor: 28°C	temp > 25 → AC ON	AC ON	Correct response
10:30 AM	Sensor: 24°C	$20 \leq \text{temp} \leq 25 \rightarrow \text{standby}$	Standby	AC off. No record it ran 30 min.
11:00 AM	Sensor: 26°C	temp > 25 → AC ON	AC ON	Turns on again. No forecast.
3:00 PM	Sensor: 31°C	temp > 25 → AC ON	AC ON	Same rule. Doesn't know it's been cycling all day.
6:00 PM	Sensor: 22°C	$20 \leq \text{temp} \leq 25 \rightarrow \text{standby}$	Standby	No learning: tomorrow same pattern.

A memory-based agent would learn: 'It heats up every afternoon. Pre-cool at 2 PM.' A reactive agent can't.

SECTION 2 — LIMITATIONS

What Reactive Agents Cannot Do

Five fundamental limitations — each one motivates the next concept

Limitation: No Memory – Concrete Failure

The agent forgets everything between interactions

Failure Scenario: Customer Support

Turn 1: User: "My order #4521 hasn't arrived!"
Agent: "Let me check... shipped, ETA tomorrow."

Turn 2: User: "OK, and my order #8890?"
Agent: "Let me check... shipped, ETA Thursday."

Turn 3: User: "Cancel the late one"
Agent: "Which order do you mean?"

→ Agent has no memory of turns 1–2.
It cannot resolve "the late one" to #4521.

What a Memory Agent Would Do

Turn 3: User: "Cancel the late one"

Memory lookup:

→ Turn 1: #4521, ETA tomorrow (LATE)
→ Turn 2: #8890, ETA Thursday (on time)

Reasoning: "The late one" = #4521

Action: cancel_order(4521)

→ Resolved without asking the user to repeat themselves

More Limitations: No Planning, Brittle, No Learning

Three more reasons reactive agents fail on complex tasks

X No Planning

Handles one trigger at a time. Cannot chain actions: 'search → compare → book → confirm' requires 4 sequential decisions. A reactive agent can only do one.

Failure: Asked to 'find the cheapest hotel near the beach with parking' — requires multi-step search, filter, rank. Reactive agent can only do a single lookup.

X Brittle

Fails on anything outside the pre-defined rule set. Novel inputs produce no action or wrong action.

Failure: User says 'I was double-charged' but rules only match 'refund'. No rule fires. Agent is silent.

X No Learning

Same response regardless of past outcomes. Never improves. Cannot adapt to user preferences.

Failure: Same thermostat cycles AC on/off every day. Never learns the afternoon pattern. Day 365 = Day 1.

SECTION 3 — WHEN TO USE

When Reactive Agents Win

Not every problem needs a reasoning loop — sometimes IF→THEN is the right answer

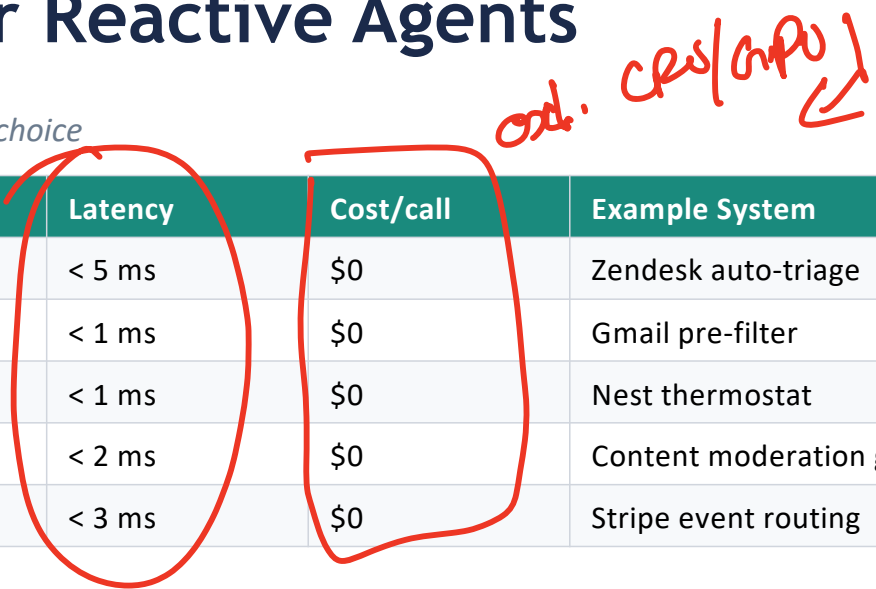
Decision Framework: Reactive vs. Full Agent

Use reactive when all four conditions are met

Condition	Meaning	Reactive ✓	Needs Full Agent ✓
Fixed categories	The input space is known and enumerable. No ambiguity. Every possible input maps to a known action.	<u>Intent routing with 5 fixed categories</u>	<u>Open-ended customer queries</u>
No chaining needed	The task completes in a single step. No need to combine results from multiple actions.	<u>Spam filter: match pattern → delete</u>	<u>Research: search → read → synthesize</u>
Speed is critical	Response must be < 10ms. Cannot afford LLM inference latency (~200-800ms).	<u>IoT sensor triggers, webhooks</u>	<u>Complex analysis, report generation</u>
\$0 cost target	Cannot afford per-call LLM costs. Rules run on CPU with zero marginal cost.	<u>Pre-filtering 100K messages/day</u>	<u>Personalized responses to each user</u>

Production Use Cases for Reactive Agents

Five real deployments where reactive agents are the right choice

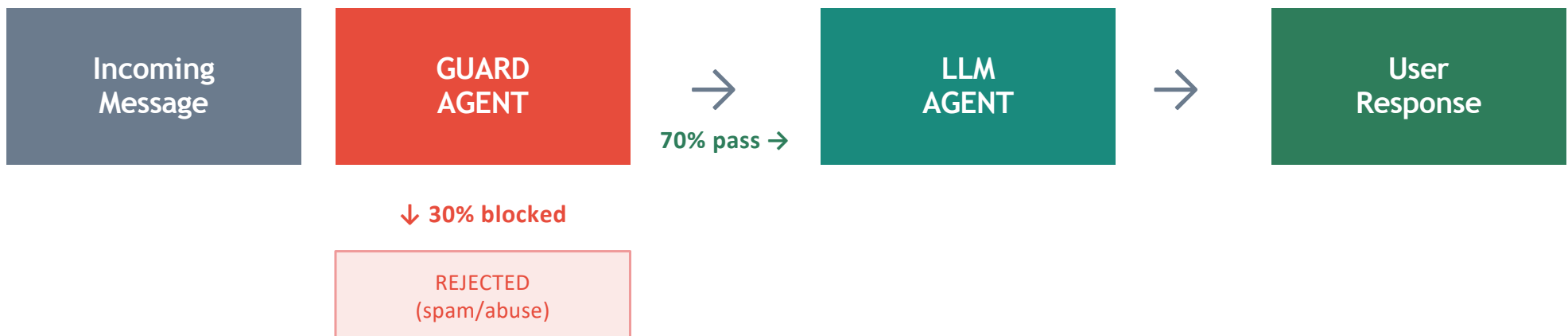


Use Case	Why Reactive Works	Latency	Cost/call	Example System
Intent Routing	Fixed categories, no ambiguity	< 5 ms	\$0	Zendesk auto-triage
Spam Filtering	Keyword + regex match	< 1 ms	\$0	Gmail pre-filter
IoT Triggers	Threshold-based (temp, motion)	< 1 ms	\$0	Nest thermostat
Guard Agents	Block banned words before LLM	< 2 ms	\$0	Content moderation gate
Webhook Dispatch	Event type → handler mapping	< 3 ms	\$0	Stripe event routing

All five cases share: fixed inputs, single-step, speed-critical, zero LLM cost.

Guard Agent Architecture

A reactive filter sitting before the LLM in the pipeline



- The guard agent is purely reactive: regex patterns, keyword blocklists, rate limits. No LLM needed.
- It processes 100K messages/day at < 2ms each with zero marginal cost.
- Only the 70% that pass reach the expensive LLM agent.

Guard Agent Cost Savings: Worked Calculation

Zero compute cost, \$54,750/year saved

Scenario: E-commerce support platform, 100,000 incoming messages per day

Without Guard Agent:

All 100K messages hit the LLM agent

$100,000 \times \$0.005/\text{call} = \$500/\text{day} = \$182,500/\text{year}$

With Guard Agent:

Guard filters 30% (spam, abuse, duplicate) → 30,000 blocked at \$0/call

Remaining 70,000 hit LLM → $70,000 \times \$0.005 = \$350/\text{day} = \$127,750/\text{year}$

Annual Savings: \$182,500 - \$127,750 = \$54,750/year

Guard agent compute cost: \$0 (CPU regex, no GPU/API calls)

ROI: Infinite (zero cost, \$54K saved)

Key Takeaways

The essential points from this concept

- Reactive agents: action = rules(percept).** No state, no planning, no learning.
- Five limitations:** no memory, no planning, brittle, no learning, no composability.
- Still valuable in production:** guard agents, IoT, routing, spam filters — fast and free.
- Guard agents save \$50K+/year** by pre-filtering before the LLM.
- Use reactive when:** fixed categories, single-step, speed-critical, \$0 cost target.

Next: AI-AG-FN-TH-000004 — Agents with Memory (adding state transforms everything)

AI-AG-FN-TH-000003 Complete

Simple Reactive Agents

Concept 3 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi